ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ РЕСПУБЛИКИ ХАКАСИЯ
«ХАКАССКИЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

ОТЧЕТ

по учебной практике
по профессиональному модулю

ПМ.02 ОСУЩЕСТВЛЕНИЕ ИНТЕГРАЦИИ ПРОГРАММНЫХ МОДУЛЕЙ

специальности 09.02.07 Информационные системы и программирование
Квалификация Технический писатель

Студент гр. ИС(ТП)-31 _____ Литвинов Н.П.
подпись *Фамилия И.О.*

Руководитель
практики
от ГБПОУ РХ ХПК _____ _____ _____ Черкашин Д.С.
оценка *дата* *подпись* *Фамилия И.О.*

Абакан 2026 г

СОДЕРЖАНИЕ

Введение 3
1 Анализ предметной области 4
1.1 Общая характеристика и современное состояние 4
1.2 Проблематика и обоснование 4
1.3 Ключевые процессы и участники системы 4
1.4 Сравнительный анализ существующих решений 5
1.5 Выводы и формулировка задач учебной практики 6
2 Построение функциональной архитектуры программного средства 7
2.1 Построение ER-диаграммы 7
2.2 Построение диаграммы Вариантов использования и диаграммы последовательности 9
2.3 Построение диаграммы Деятельности, диаграммы Состояний, диаграммы Классов 11
2.4 Построение диаграмм потоков данных 14
3 Разработка прототипа 17
3.1 Разработка макета интерфейса в Figma 17
4 Тестирование и оценка качества 19
4.1 Изучение работы в системе контроля версий, настройка 19
4.2 Разработка тестового сценария 20
4.3 Разработка тестовых пакетов 23
4.4 Оценка программных средств с помощью метрик 25
Заключение 26
Список использованных источников 28
Глоссарий 29
Список аббревиатур 32
Приложение А: Техническое задание 34

ВВЕДЕНИЕ

В условиях стремительной цифровизации коммуникационных процессов владение современными инструментами обмена данными является ключевым компетенцией специалистов в области разработки программного обеспечения. Мессенджеры представляют собой сложные распределённые системы, требующие знаний в области клиент-серверного взаимодействия, проектирования баз данных, обеспечения безопасности и оптимизации производительности.

Цель работы – провести комплексный анализ архитектуры и функционала мессенджера Telegram, разработать полный комплект проектной и тестовой документации (модели данных, UML/DFD-диаграммы, прототип интерфейса, чек-листы, тестовые пакеты), а также оценить качество программного средства.

Для достижения цели необходимо решить следующие задачи:

- Провести анализ предметной области и сформулировать требования к разрабатываемому программному средству
- Спроектировать логическую структуру базы данных и построить ER-диаграмму.
- Разработать функциональную и архитектурную модели системы с использованием нотаций UML и DFD
- Создать интерактивный макет пользовательского интерфейса в Figma
- Настроить систему контроля версий Git для ведения проектной документации и кода
- Разработать чек-лист (≥70 пунктов) и тестовые пакеты (≥3 пакета) для проверки основных сценариев
- Оценить качество программного средства с помощью программных метрик и оформить отчётную документацию

Объект исследования: Мессенджер Telegram как высоконагруженное клиент-серверное приложение для мгновенного обмена данными.

Предмет исследования: Функциональная архитектура, процессы взаимодействия компонентов, пользовательские сценарии и показатели качества программного обеспечения мессенджера Telegram.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Общая характеристика и современное состояние

В условиях повсеместной цифровизации коммуникационных процессов мессенджеры стали базовым инструментом повседневного взаимодействия как в бытовой, так и в профессиональной среде. Современные системы мгновенного обмена сообщениями представляют собой сложные распределённые приложения, реализующие клиент-серверную архитектуру, работу с протоколами реального времени (WebSocket, MQTT), реляционными и NoSQL базами данных, а также механизмы криптографической защиты данных.

Мессенджер Telegram — это кроссплатформенное облачное программное обеспечение для мгновенного обмена сообщениями и мультимедийными файлами, разработанное компанией Telegram FZ-LLC. Приложение поддерживает работу на мобильных устройствах (iOS, Android), десктопных ОС (Windows, macOS, Linux) и в веб-интерфейсе, обеспечивая синхронизацию данных между устройствами через облачную инфраструктуру.

1.2 Проблематика и обоснование

Разработка данного прототипа обусловлена необходимостью закрепления на практике навыков полного цикла создания ПО: от сбора требований и проектирования архитектуры до тестирования и оценки качества. В образовательном процессе часто не хватает комплексных проектов, позволяющих одновременно отработать:

- Проектирование нормализованных схем БД
- Реализацию асинхронных коммуникаций (WebSocket/Push)
- Настройку аутентификации и авторизации
- Создание адаптивных UI-макетов
- Написание тестовой документации и расчёт метрик кода

1.3 Ключевые процессы и участники системы

В рамках предметной области выделяются следующие роли и сценарии взаимодействия в таблице 1:

Таблица 1 – Сценарии взаимодействия

Роль	Описание	Основные сценарии
Пользователь	Физическое лицо, использующее клиентское приложение для коммуникации	Регистрация, авторизация, управление контактами, создание чатов, отправка/получение сообщений и вложений, изменение настроек профиля
Сервер Telegram	Распределённая инфраструктура обработки запросов	Аутентификация через SMS/звонок, маршрутизация сообщений, хранение облачных данных, синхронизация между устройствами
Бот / Интеграция	Автоматизированный агент, взаимодействующий через Bot API	Обработка команд, рассылка уведомлений, интеграция с внешними сервисами

Типовой жизненный цикл сообщения

- Клиент шифрует сообщение и отправляет запрос на сервер через MTProto
- Сервер аутентифицирует запрос, проверяет права доступа к чату
- Сообщение сохраняется в облачном хранилище и транслируется получателям через push-уведомления или активное соединение
- Статусы доставки (✓ → ✓✓ → ✓✓ синие) обновляются на стороне отправителя
- Получатель открывает чат — сообщение помечается как прочитанное, данные синхронизируются между устройствами

1.4 Сравнительный анализ существующих решений

Для обоснования архитектуры прототипа был проведён анализ трёх наиболее распространённых аналогов:

Таблица 2 – Сравнение

Критерий	**Telegram**	**WhatsApp**	**Signal**
Архитектура	Облачная, MTProto	Клиент-сервер, Signal Protocol	Полностью E2E, Signal Protocol
Шифрование	Серверное (по умолчанию)	E2E (только в секретных чатах)	E2E (всегда)
Хранение файлов	Облачное, до 2 ГБ	Локальное/облачное (до 100 МБ)	Локальное (до 100 МБ)
Боты / API	Полноценный Bot API	Ограниченный	Нет

Продолжение таблицы 2

Поддержка тестирования	Проприетарные инструменты	Нет публичных SDK	Нет публичных SDK
Облачное хранение	Да (неограниченно)	Ограничено	Нет (только локально)
Открытость клиента	Да (GPL)	Нет	Да (GPL)

Вывод: Telegram сочетает высокую функциональность, масштабируемость и открытость клиентской части, что делает его удобным объектом для учебного анализа архитектуры и взаимодействия компонентов.

1.5 Выводы и формулировка задач учебной практики

Проведённый анализ подтверждает, что исследование мессенджера Telegram соответствует целям практики и позволяет отработать компетенции в области:

- анализа требований и функционала сложных программных систем
- моделирования архитектуры и процессов с использованием UML/DFD
- тестирования функциональности и оценки качества интерфейса

Для достижения цели практики необходимо решить следующие задачи:

- Построить ER-модель данных, абстрагируясь от внутренней реализации, на основе наблюдаемого поведения системы
- Разработать диаграммы: Use Case (варианты использования), Sequence (последовательность отправки сообщения), Activity (алгоритм регистрации), Class (логическая структура сущностей)
- Создать прототип интерфейса в Figma, воспроизводящий ключевые экраны Telegram (список чатов, диалог, настройки)
- Настроить систему контроля версий (Git), зафиксировать этапы работы в репозитории.
- Разработать чек-лист (≥70 пунктов) и тестовые пакеты (≥3) для проверки основных сценариев использования

Объект исследования: программный комплекс мессенджера Telegram как пример высоконагруженного клиент-серверного приложения.

Предмет исследования: функциональная архитектура, процессы взаимодействия компонентов и пользовательские сценарии мессенджера Telegram в контексте учебной практики по ПМ.02.

2 ПОСТРОЕНИЕ ФУНКЦИОНАЛЬНОЙ АРХИТЕКТУРЫ ПРОГРАММНОГО СРЕДСТВА

В данном разделе проводится моделирование функциональной архитектуры разрабатываемого мессенджера «Telegram» с использованием нотаций UML и IDEF (DFD). В качестве CASE-средства выбран бесплатный кроссплатформенный инструмент draw.io, что обусловлено его наглядностью, поддержкой экспорта в векторные форматы и достаточным функционалом для построения учебных диаграмм в рамках временных рамок практики.

2.1 Построение ER-диаграммы

ER-диаграмма на рисунке 1 описывает логическую структуру данных, используемых в мессенджере. На диаграмме представлены пять основных сущностей. Модель приведена к третьей нормальной форме для минимизации избыточности данных.

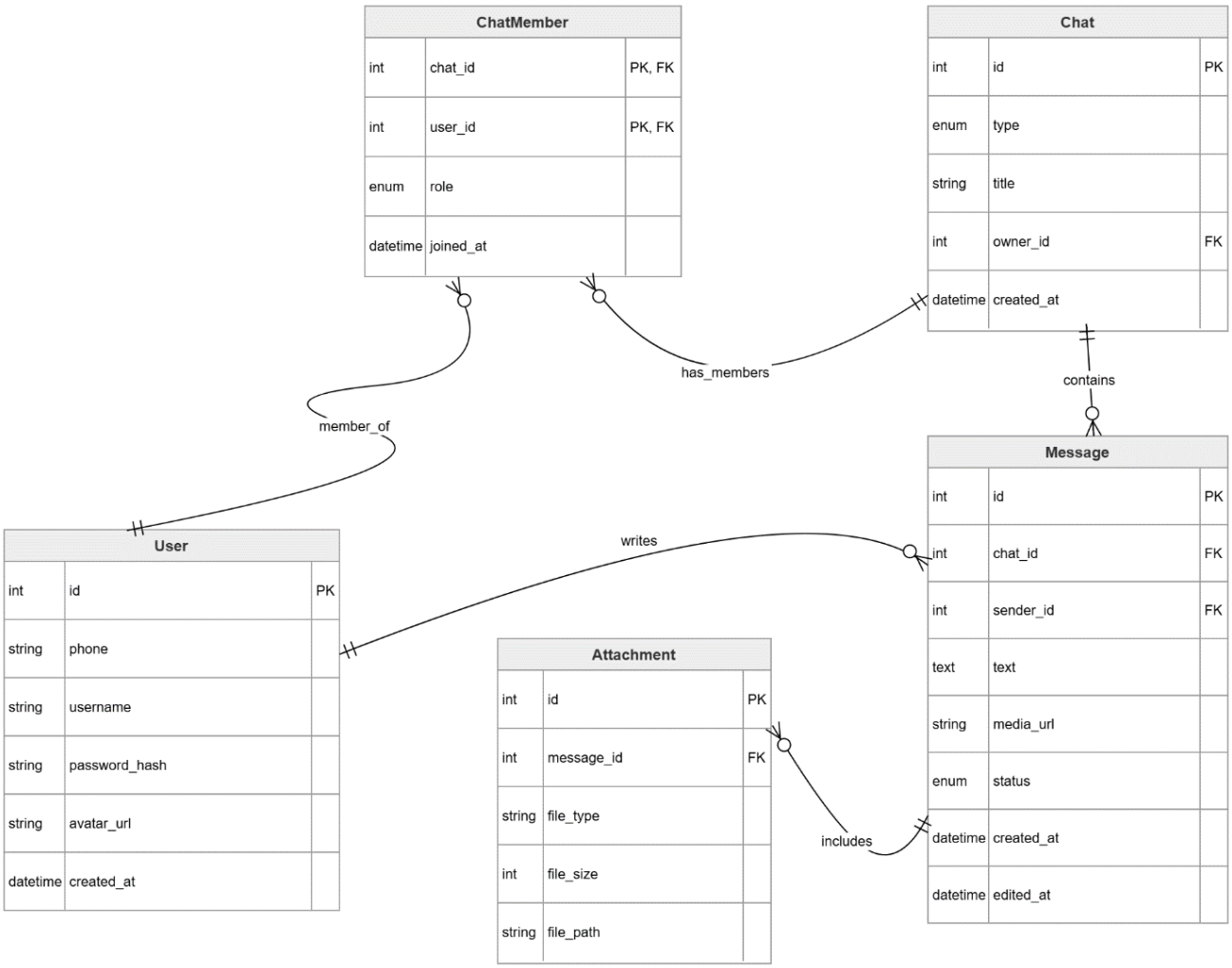

Рисунок 1 – ER-диаграмма базы данных мессенджера «Telegram»

Основные связи:

- User → Message: один ко многим (пользователь пишет сообщения)
- Chat → Message: один ко многим (чат содержит сообщения)
- User ↔ Chat: многие ко многим (реализуется через сущность ChatMember)
- Message → MediaAttachment: один ко многим (сообщение может содержать вложения)
- Message → Reaction: один ко многим (на сообщение могут быть реакции)

Модель нормализована до 3НФ для минимизации избыточности. Индексация полей chat_id, user_id, created_at обеспечивает высокую производительность при выборке истории сообщений и синхронизации между устройствами.

2.2 Построение диаграммы Вариантов использования и диаграммы последовательности

Диаграмма Вариантов использования (Use Case)

Актеры системы:

- Пользователь - основной актер, инициирующий сценарии использования
- Сервер Telegram - вторичный актер, обрабатывающий запросы и обеспечивающий доставку данных
- Контакт - пассивный актер, получатель сообщений

Таблица 1 – Основные варианты использования

№	**Вариант использования**	**Описание**	**Включение (Include)**
UC-01	Регистрация / Авторизация	Вход в систему по номеру телефона и коду из SMS	UC-01.1 Проверка номера
UC-02	Управление контактами	Добавление, удаление, поиск контактов	—
UC-03	Создание чата	Инициация личного или группового диалога	UC-02
UC-04	Отправка сообщения	Передача текста/медиа в чат	UC-04.1 Шифрование, UC-04.2 Загрузка медиа
UC-05	Получение сообщения	Отображение входящих данных	—
UC-06	Управление настройками	Изменение профиля, приватности, уведомлений	—
UC-07	Работа с медиа	Просмотр, скачивание, пересылка файлов	—

Диаграмма вариантов использования (Use Case) на рисунке 2 отображает функциональные требования к системе с точки зрения внешних актёров. На диаграмме представлены три основных актёра. Диаграмма включает семь основных вариантов использования.

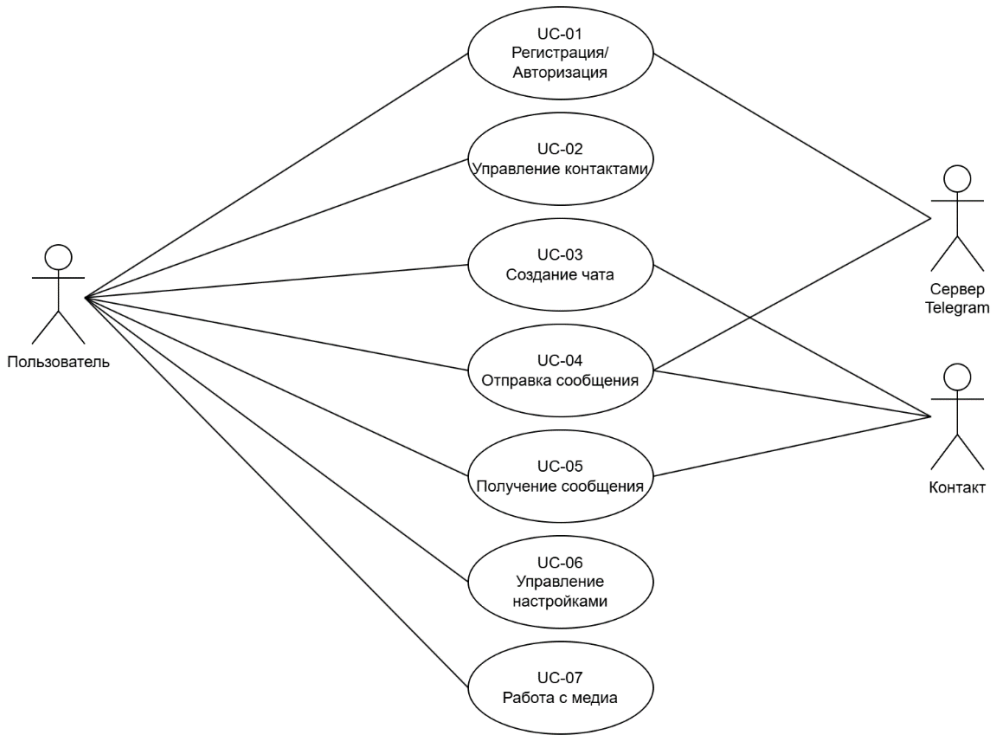

Рисунок 2 – Диаграмма вариантов использования (Use Case)

Диаграмма последовательности (Sequence) — сценарий «Отправка сообщения»

Участники: User_A (Отправитель), Client_App, Telegram_Server, Database, User_B (Получатель).

Последовательность действий:

- User_A вводит текст и нажимает «Отправить»
- Client_App валидирует ввод, шифрует сообщение (MTProto)
- Client_App отправляет POST/WS-запрос на Telegram_Server с токеном авторизации
- Telegram_Server проверяет токен и права доступа к чату
- Telegram_Server сохраняет сообщение в Database с статусом sent
- Telegram_Server отправляет подтверждение Client_App (статус delivered)
- Telegram_Server транслирует сообщение Client_App получателя (User_B) через push/WS
- Client_App получателя отображает уведомление и сохраняет сообщение локально
- При открытии чата Client_App отправляет статус read на сервер
- Telegram_Server обновляет статус в БД и уведомляет User_A

Диаграмма последовательности (Sequence) на рисунке 3 детализирует временное взаимодействие компонентов системы при отправке сообщения. Диаграмма показывает десять последовательных шагов: от ввода текста отправителем до обновления статуса прочтения в базе данных. Особое внимание уделено процессам шифрования (MTProto), проверки токена авторизации и транслирования сообщения через WebSocket/Push-уведомления.

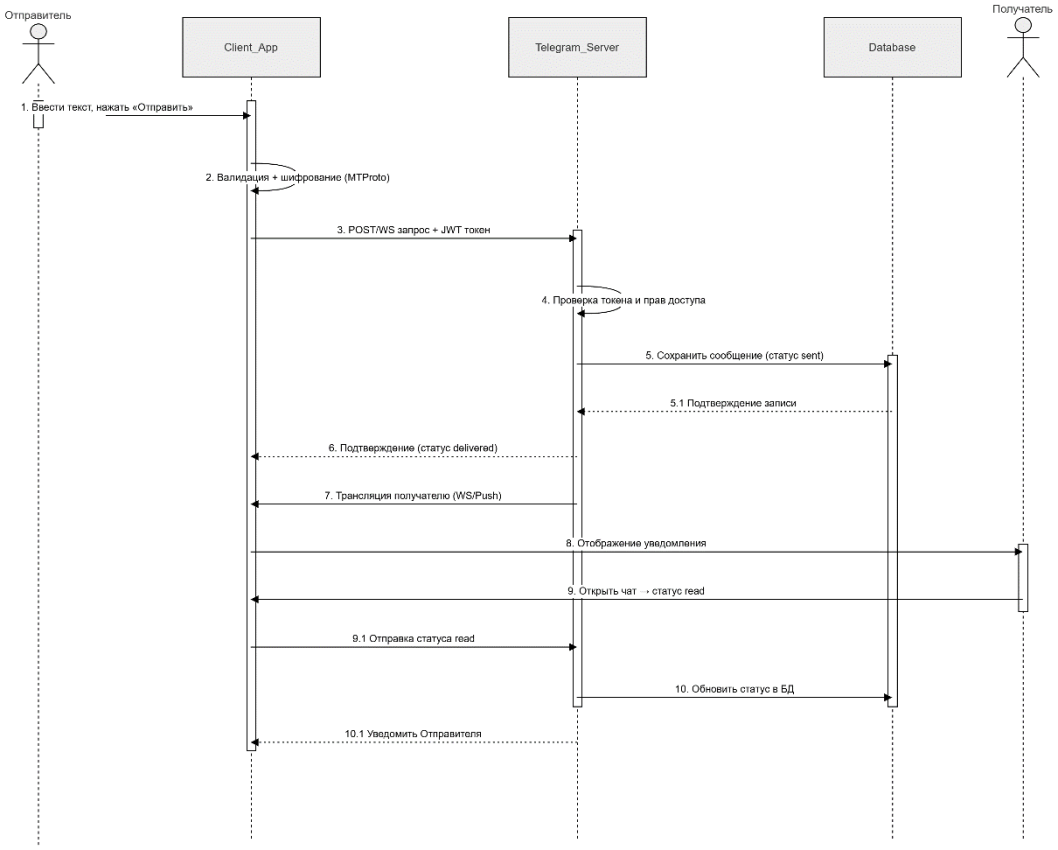

Рисунок 3 – Диаграмма последовательности «Отправка сообщения»

2.3 Построение диаграммы Деятельности, диаграммы Состояний, диаграммы Классов

Диаграмма деятельности (Activity) на рисунке 4 описывает алгоритм первичного входа в систему и создания сессии пользователя. Процесс начинается с ввода номера телефона и отправки запроса на сервер для генерации OTP-кода. Диаграмма включает условную проверку корректности введённого кода: при успешной валидации создаётся сессия с генерацией JWT-токена, при ошибке — система

допускает до трёх попыток ввода. После исчерпания попыток применяется временная блокировка для защиты от подбора кода.

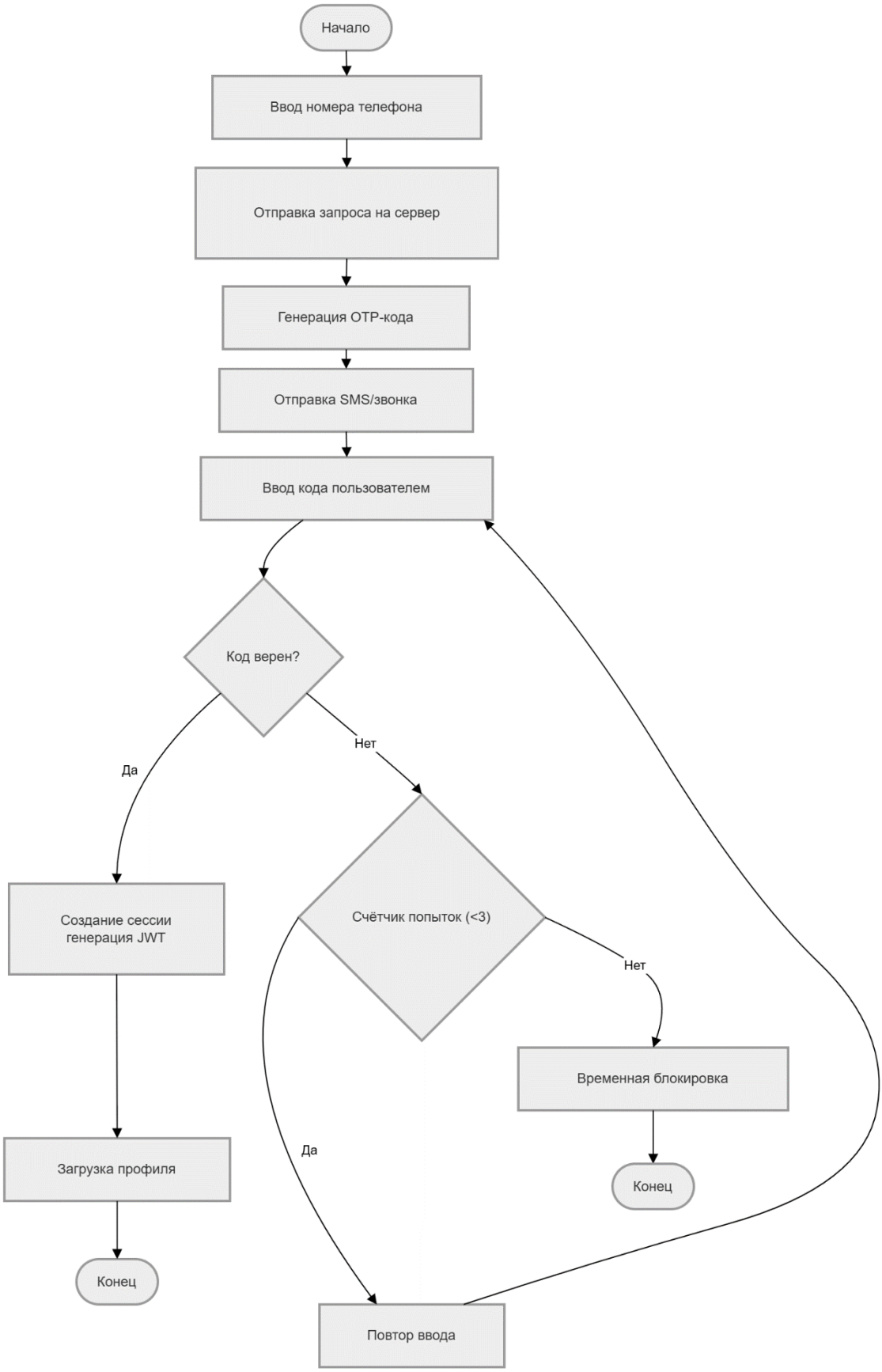

Рисунок 4 – Диаграмма деятельности «Регистрация пользователя»

Диаграмма состояний на рисунке 5 отражает поведение объекта «Сообщение» на протяжении всего срока его существования в системе. Процесс начинается с создания черновика на стороне клиента. После нажатия кнопки отправки сообщение переходит в состояние отправки, а затем ожидает подтверждения от сервера Telegram.

По мере продвижения пакета данных по сети, сообщение проходит через состояния «Отправлено» (сохранено на сервере), «Доставлено» (получатель получил уведомление) и «Прочитано» (получатель открыл диалог). Диаграмма также предусматривает альтернативные переходы, например, удаление сообщения отправителем или получателем, которое переводит объект в конечное состояние.

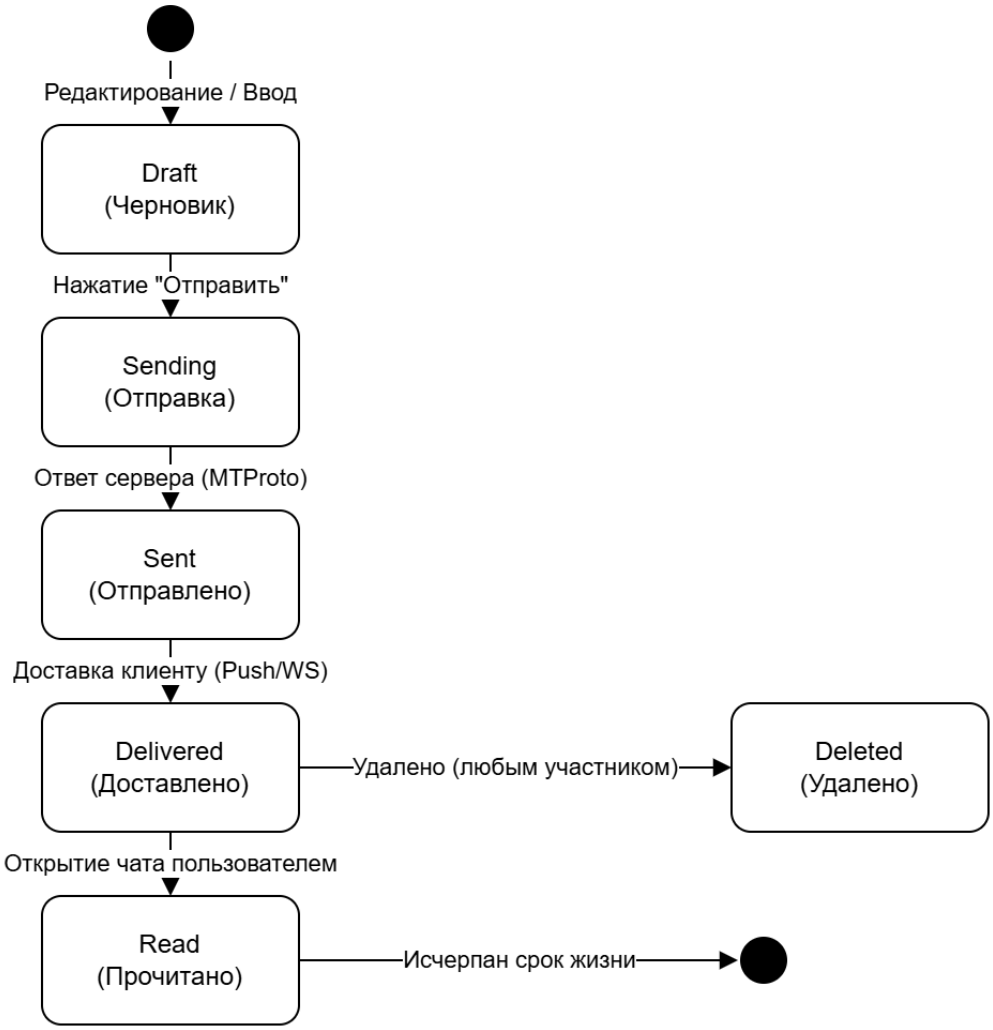

Рисунок 5 – Диаграмма состояний сообщения

Диаграмма классов (Class) на рисунке 6 отражает объектно-ориентированную структуру разрабатываемого программного обеспечения. Модель включает три логических слоя: слой данных (User, Chat, Message, Attachment), сервисный слой (AuthService, MessageService, FileSyncManager, WebSocketHandler) и клиентский слой (UIMainWindow, ChatListController, MessageInputView, AuthManager). Между классами установлены отношения агрегации (AuthService агрегирует AuthManager), ассоциации (MessageService связана с DatabaseRepository) и реализации паттерна Observer (WebSocketHandler отслеживает изменения в MessageService). Такая архитектура обеспечивает разделение ответственности и упрощает поддержку кода.

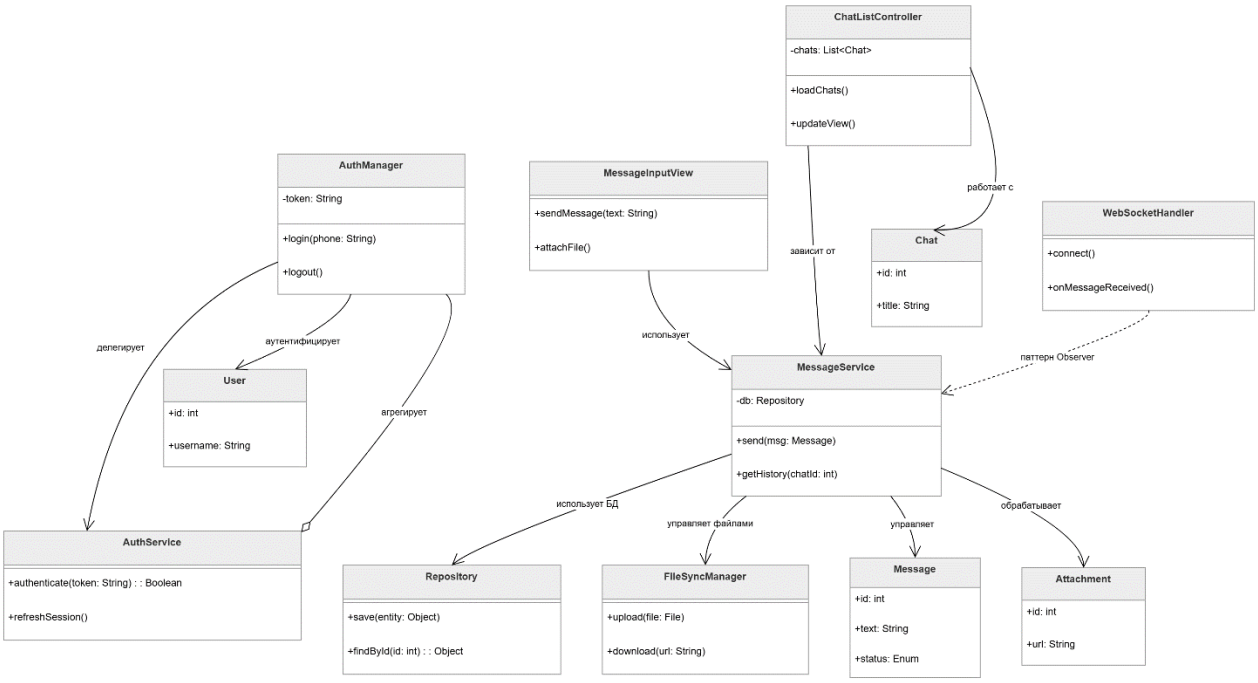

Рисунок 6 – Диаграмма классов клиент-серверного модуля

2.4 Построение диаграмм потоков данных

Диаграммы потоков данных (DFD) детализируют движение информации внутри системы без привязки к физической реализации компонентов.

Контекстная диаграмма потоков данных (DFD Уровень 0) на рисунке 7 описывает систему как единый процесс «0.0 Мессенджер Telegram» и её взаимодействие с внешними сущностями.

Описывает систему как единый процесс 0.0 Мессенджер Telegram.

Внешние сущности: Пользователь, SMS-провайдер, CDN-хранилище.

Основные потоки: Запросы аутентификации, сообщения, медиафайлы, push-уведомления, статусы доставки.

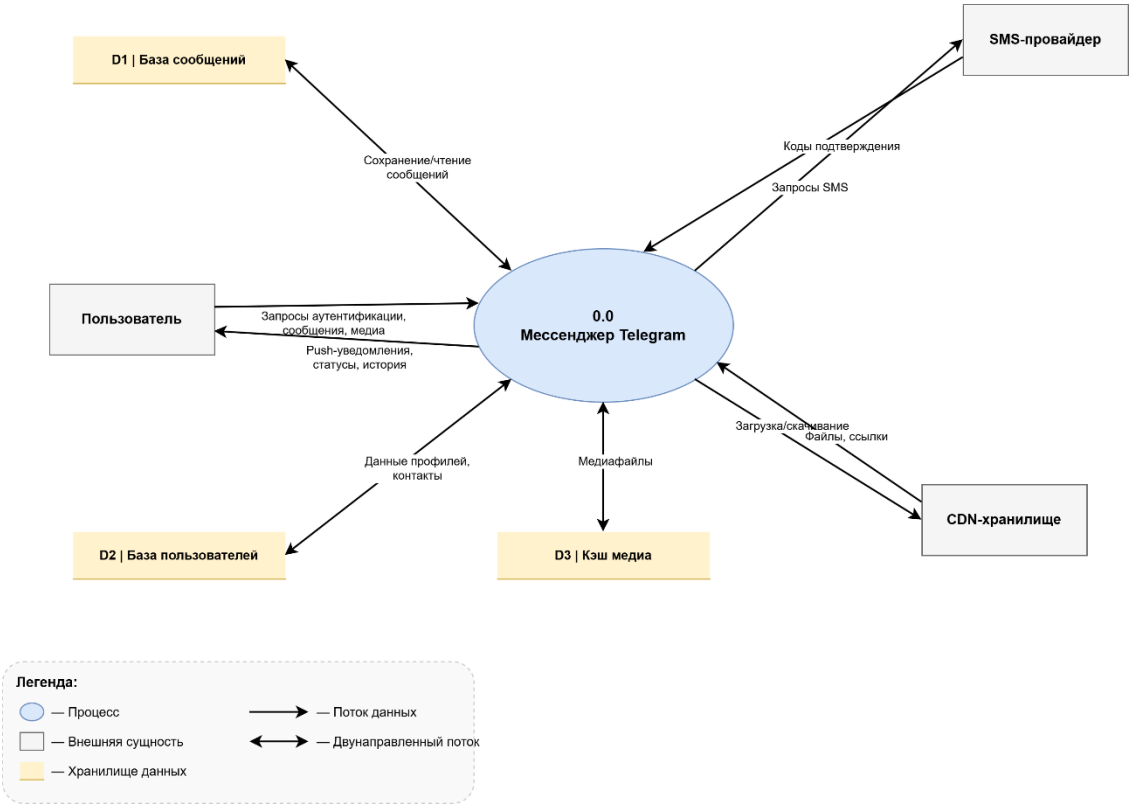

Рисунок 7 – DFD Уровень 0 (Контекстная диаграмма)

Диаграмма потоков данных первого уровня (DFD Уровень 1) на рисунке 8 представляет декомпозицию системы на пять логических процессов.Система разбита на 5 логических процессов:

- Аутентификация и управление сессиями
- Управление чатами и участниками
- Маршрутизация сообщений
- Обработка и хранение медиа
- Синхронизация и уведомления

Хранилища данных: D1 Пользователи, D2 Чаты, D3 Сообщения, D4 Файлы.

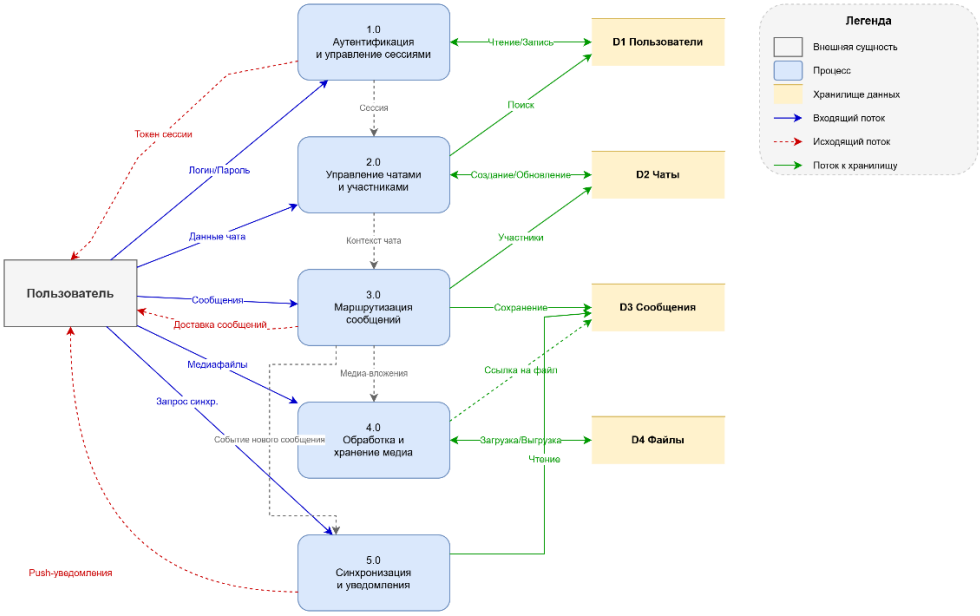

Рисунок 8 – DFD Уровень 1

Диаграмма потоков данных второго уровня (DFD Уровень 2) на рисунке 89детализирует процесс «3.0 Маршрутизация сообщений» на пять подпроцессов. Раскрывает внутреннюю логику доставки контента:

- Валидация входящего пакета (проверка формата, размера, прав доступа)
- Шифрование/Дешифрование (применение криптографических алгоритмов)
- Определение получателей (чтение таблицы ChatMember)

  Запись в БД и кэш (транзакционное сохранение в D3)
- Рассылка через WebSocket (отправка активным клиентам)

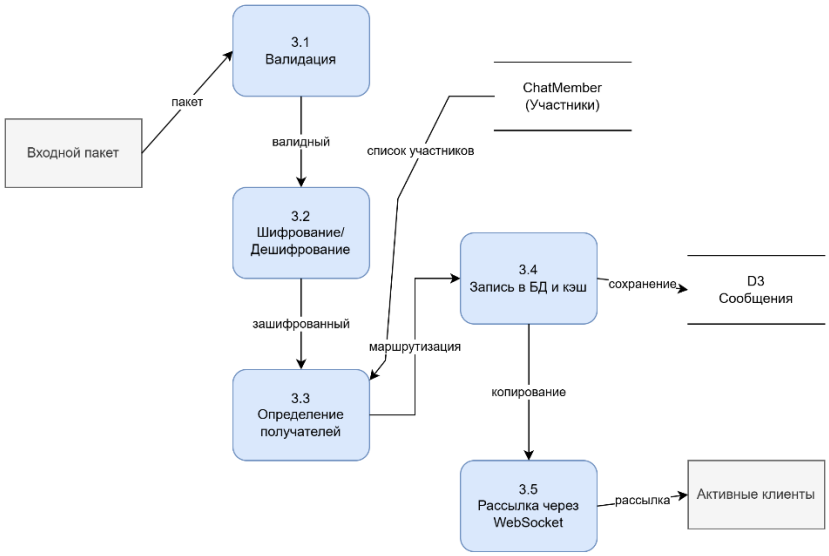

Рисунок 9 – DFD Уровень 2 (Процесс 3.0)

3 РАЗРАБОТКА ПРОТОТИПА

3.1 Разработка макета интерфейса в Figma

На рисунке 11 изображён макет главного экрана мессенджера Telegram, разработанный в инструменте Figma. Макет выполнен для платформы iOS 26 с использованием новейшего дизайна Apple «Liquid Glass», характеризующегося полупрозрачными элементами и эффектом размытия фона. Интерфейс включает: верхнюю панель поиска, список чатов с отображением последних сообщений и времени, индикаторы непрочитанных сообщений и нижнюю навигационную панель. Макет придерживается разработанной цветовой схемы, представленной на рисунке 10, и принципов адаптивности.

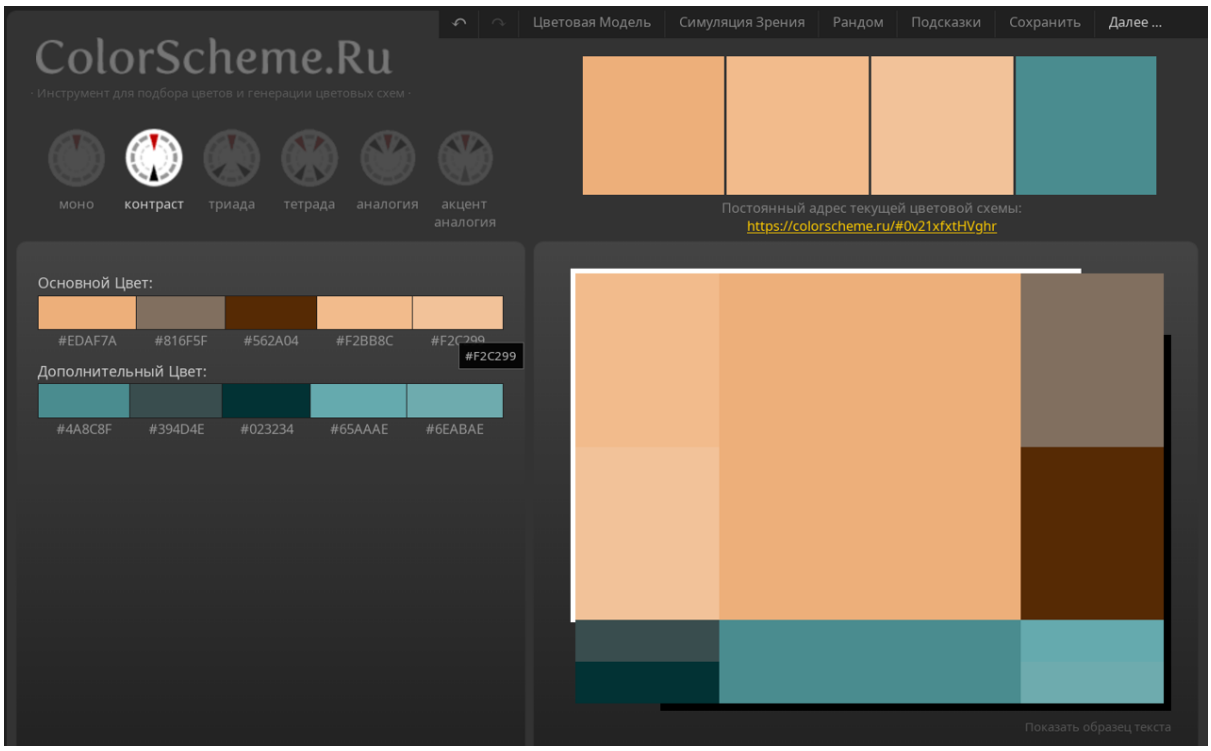

Рисунок 10 – Цветовая схема к макету

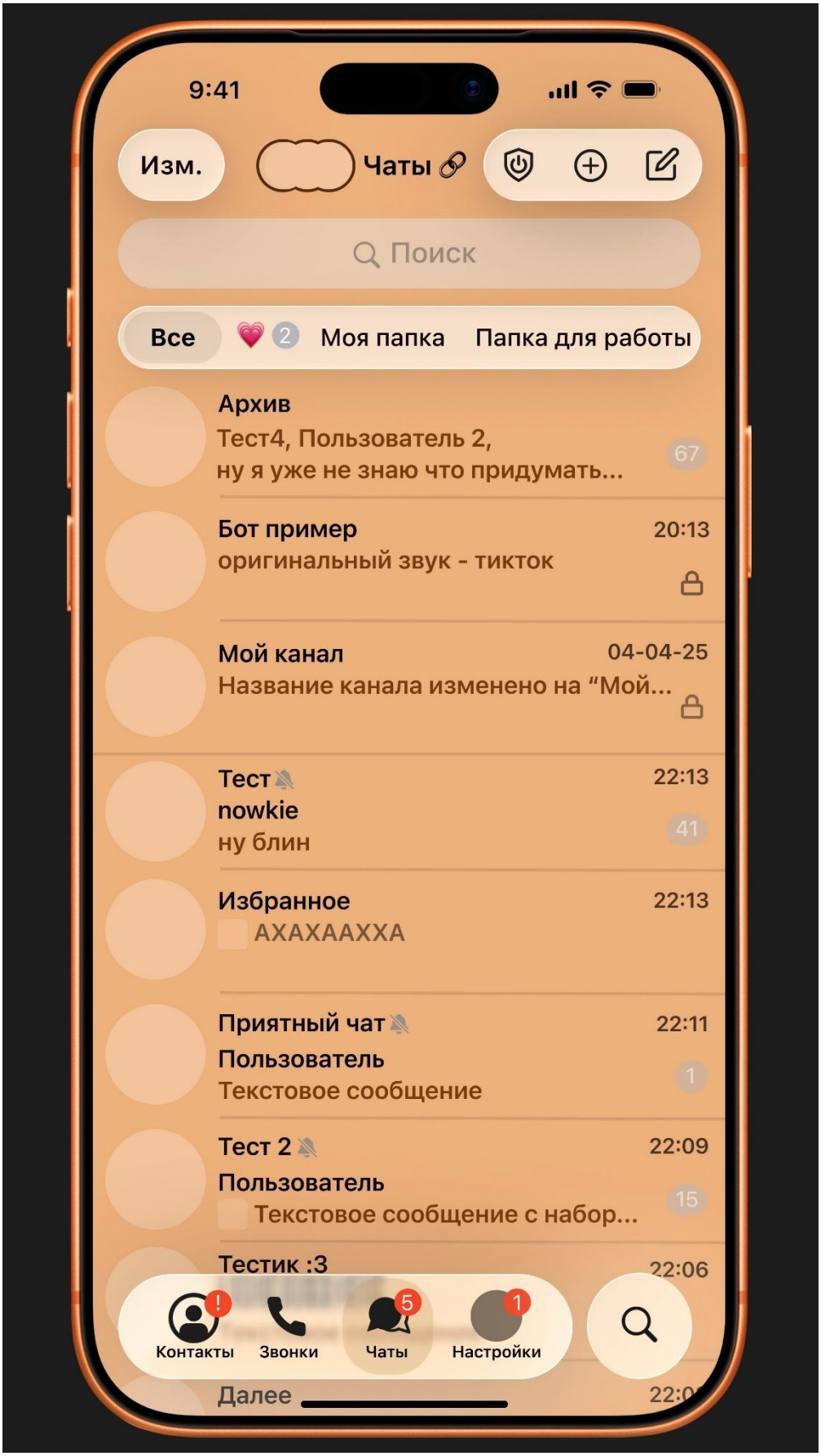

Рисунок 11 – Мессенджер Telegram, главный экран

4 ТЕСТИРОВАНИЕ И ОЦЕНКА КАЧЕСТВА

В данном разделе описывается процесс настройки системы контроля версий, разрабатываются тестовые сценарии и пакеты для проверки функционала учебного прототипа мессенджера “Telegram”, а также проводится оценка качества программного средства с использованием программных метрик. Тестирование выполняется на стороне клиентского приложения и имитации серверной логики в рамках учебной практики.

4.1 Изучение работы в системе контроля версий, настройка

Система контроля версий (СКВ) Git представляет собой распределённую систему учёта изменений исходного кода, позволяющую фиксировать состояние проекта на различных этапах разработки, создавать изолированные ветки для экспериментов и обеспечивать совместную работу над репозиторием.

На рисунке 12 представлен экран входа для подключения консоли к GitHub с целью создания открытого репозитория: “https://github.com/litvnik/telegram-project”. Репозиторий будет содержать отчёт, изображения диаграмм, а также презентацию.

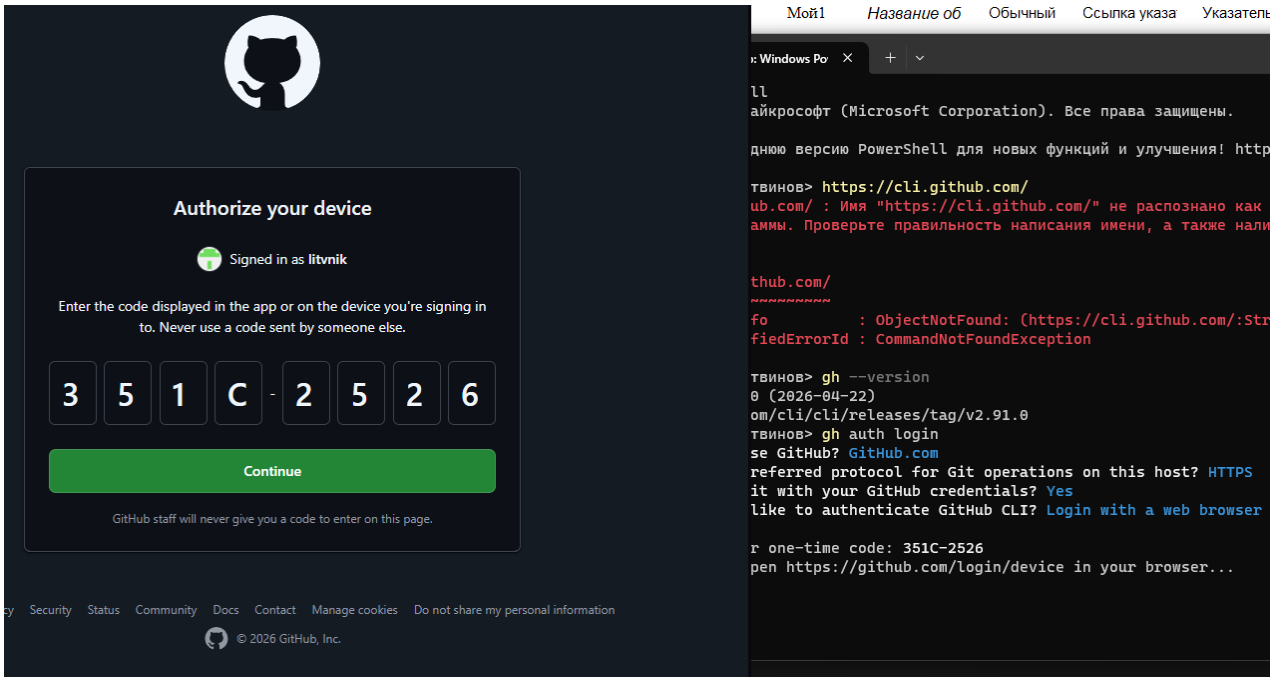

Рисунок 12 – GitHub

Ветвление (Branching): Используется упрощённая модель Git Flow:

- main – стабильная версия, готовая к сдаче

- develop – основная ветка разработки
- feature/auth, feature/chat, feature/media – ветки для реализации отдельных модулей

Фиксация изменений: Коммиты выполняются с соблюдением конвенции Conventional Commits:

- feat: добавить авторизацию по номеру телефона
- fix: исправить вёрстку списка чатов
- docs: обновить README с инструкцией по запуску

Синхронизация с удалённым репозиторием: Настройка связи с GitHub, выполнение push, создание Pull Request для проверки кода перед мержем в main.

Использование Git обеспечивает прозрачность истории изменений, возможность отката к стабильным состояниям (git checkout, git reset) и формирует навык работы в современных DevOps-процессах.

4.2 Разработка тестового сценария

Чек-лист представляет собой структурированный список проверок, охватывающий основной функционал прототипа. Всего разработано 75 пунктов, сгруппированных по модулям системы.

Таблица 2 – Чек-лист проверки функционала мессенджера «Telegram»

№	**Модуль**	**Проверка**
1	Авторизация	Отображение экрана входа при первом запуске
2	Авторизация	Поле ввода номера телефона принимает только цифры
3	Авторизация	Валидация формата номера (+7, 10 цифр)
4	Авторизация	Отправка запроса на сервер при нажатии «Далее»
5	Авторизация	Отображение индикатора загрузки во время ожидания кода
6	Авторизация	Получение SMS-кода (имитация/реальный)
7	Авторизация	Ввод 6-значного OTP-кода
8	Авторизация	Успешная авторизация при верном коде
9	Авторизация	Блокировка ввода после 3 неверных попыток

Продолжение таблицы 2

10	Авторизация	Автоматический редирект на главный экран после входа
11	Профиль	Отображение аватара по умолчанию
12	Профиль	Загрузка пользовательского изображения
13	Профиль	Обрезка/масштабирование фото при загрузке
14	Профиль	Сохранение имени пользователя (username)
15	Профиль	Валидация username (латиница, цифры, подчеркивание, 5-32 символа)
16	Профиль	Уникальность username в системе
17	Профиль	Редактирование отображаемого имени (first/last name)
18	Профиль	Сохранение изменений профиля в БД
19	Профиль	Кнопка «Выйти из аккаунта» очищает локальную сессию
20	Профиль	Повторный вход требует полной авторизации
21	Чаты	Отображение пустого списка при отсутствии диалогов
22	Чаты	Кнопка создания нового чата (иконка карандаша)
23	Чаты	Поиск контакта по имени или username
24	Чаты	Создание личного диалога 1-на-1
25	Чаты	Создание группового чата (до 50 участников)
26	Чаты	Назначение администратора группы
27	Чаты	Удаление участника из группы
28	Чаты	Выход из группы
29	Чаты	Закрепление сообщения в чате
30	Чаты	Открепление сообщения
31	Чаты	Удаление чата из списка
32	Чаты	Архивирование чата
33	Чаты	Отображение непрочитанных сообщений (бейдж)
34	Чаты	Сортировка чатов по времени последнего сообщения

Продолжение таблицы 2

35	Чаты	Пагинация при загрузке старых сообщений
36	Сообщения	Поле ввода текста активируется при фокусе
37	Сообщения	Отправка текстового сообщения по кнопке/Enter
38	Сообщения	Отображение сообщения в чате отправителя
39	Сообщения	Статус «отправлено» (одна галочка)
40	Сообщения	Статус «доставлено» (две серые галочки)
41	Сообщения	Статус «прочитано» (две синие галочки)
42	Сообщения	Редактирование сообщения в течение 48 часов
43	Сообщения	Удаление сообщения «для себя»
44	Сообщения	Удаление сообщения «для всех»
45	Сообщения	Ответ на конкретное сообщение (Reply)
46	Сообщения	Пересылка сообщения в другой чат
47	Сообщения	Копирование текста сообщения в буфер обмена
48	Сообщения	Поиск по тексту в текущем чате
49	Сообщения	Подсветка найденных фрагментов
50	Сообщения	Автопрокрутка к новым сообщениям
51	Сообщения	Корректное отображение эмодзи и Unicode
52	Медиа	Выбор файла из галереи/файлового менеджера
53	Медиа	Отображение превью изображения перед отправкой
54	Медиа	Загрузка фото (до 5 МБ)
55	Медиа	Загрузка документа (до 10 МБ)
56	Медиа	Проверка поддерживаемых форматов (JPEG, PNG, PDF, TXT)
57	Медиа	Отображение прогресса загрузки
58	Медиа	Обработка ошибки при превышении размера файла
59	Медиа	Скачивание вложения получателем

Продолжение таблицы 2

60	Медиа	Открытие скачанного файла во внешнем просмотрщике
61	Уведомления	Включение/выключение звуковых оповещений
62	Уведомления	Отображение push-уведомления в фоне
63	Уведомления	Переход в чат по клику на уведомление
64	Уведомления	Бесшумный режим (Mute) для конкретного чата
65	Безопасность	Хранение токена сессии в защищённом хранилище
66	Безопасность	Автоматический выход при истечении срока токена
67	Безопасность	Блокировка экрана при сворачивании приложения (опция)
68	Настройки	Переключение светлой/тёмной темы
69	Настройки	Сохранение выбранной темы в локальных настройках
70	Настройки	Изменение языка интерфейса
71	Настройки	Сброс к настройкам по умолчанию
72	Настройки	Отображение версии приложения и сборки
73	Производительность	Время загрузки списка чатов < 2 сек
74	Производительность	Плавность анимаций при скролле
75	Производительность	Отсутствие утечек памяти при длительном использовании

4.3 Разработка тестовых пакетов

Тестовые пакеты представляют собой наборы структурированных тест-кейсов, объединённых по функциональному признаку. Разработано 3 пакета по 4 тест-кейса каждый.

Таблица 3 – Пакет 1: Аутентификация и управление сессиями

ID	**Название**	**Предусловия**	**Шаги выполнения**	**Ожидаемый результат**
TC-01.01	Успешная авторизация по номеру телефона	Приложение установлено, сеть доступна	1. Ввести валидный номер 2. Получить код (имитация) 3. Ввести верный код	Переход на главный экран, сессия сохранена

Продолжение таблицы 3

TC-01.02	Блокировка при неверном коде	Номер введён, ожидание кода активно	1. Ввести неверный код 3 раза подряд	Поле ввода блокируется на 30 сек, отображается предупреждение
TC-01.03	Выход из аккаунта	Пользователь авторизован	1. Открыть Настройки → Выйти	Сброс локальных данных, редирект на экран входа
TC-01.04	Восстановление сессии при перезапуске	Активная сессия, приложение закрыто	1. Открыть приложение заново	Автоматический вход без повторного ввода кода

Таблица 4 – Пакет 2: Обмен сообщениями в личных чатах

ID	**Название**	**Предусловия**	**Шаги выполнения**	**Ожидаемый результат**
TC-02.01	Отправка текста	Создан личный чат	1. Ввести текст "Тест" 2. Нажать "Отправить"	Сообщение появляется в чате, статус sent → delivered
TC-02.02	Редактирование текста	Отправлено <48ч назад	1. Долгое нажатие → Редактировать	Поле ввода заполняется старым текстом, кнопка меняется на "Сохранить"
TC-02.03	Удаление "для всех"	Отправлено сообщение	1. Выбрать "Удалить для всех"	Сообщение исчезает у обоих участников, остаётся заглушка
TC-02.04	Статусы прочтения	Получатель открыл чат	1. Отправить сообщение получателю	Галочки меняются с серых на синие у отправителя

Таблица 5 – Пакет 3: Работа с медиафайлами

ID	**Название**	**Предусловия**	**Шаги выполнения**	**Ожидаемый результат**
TC-03.01	Загрузка изображения	Выбран файл <5МБ (JPEG)	1. Нажать скрепку → Галерея 2. Выбрать фото → Отправить	Превью в чате, статус uploaded, файл доступен для скачивания
TC-03.02	Превышение размера файла	Выбран файл >5МБ	1. Выбрать крупный файл → Отправить	Поле ввода заполняется старым текстом, кнопка меняется на "Сохранить"
TC-03.03	Скачивание документа	В чате есть вложение	1. Нажать на файл → Скачать	Файл сохраняется в папку Downloads, открывается просмотрщик
TC-03.04	Пересылка медиа	Выбрано изображение в чате	1. Долгое нажатие → Переслать → Выбрать чат	Медиа появляется в целевом чате с пометкой "Переслано"

4.4 Оценка программных средств с помощью метрик

Оценка качества кода и архитектуры учебного прототипа выполнена с использованием статических и динамических метрик. Анализ проводился инструментами ESLint/Pylint (в зависимости от стека), Jest/PyTest и SonarQube.

Основные используемые метрики:

- Цикломатическая сложность (McCabe) – измеряет количество линейно независимых путей в коде. Рассчитывается по формуле:
- M = E - N + 2P

  где E – количество рёбер графа, N – количество вершин, P – число компонент связности.
- Норма: ≤10 на функцию/метод. Значение выше указывает на необходимость рефакторинга.
- Объём Холстеда (Halstead Volume) – оценивает сложность программы через операторы и операнды:
- Интерпретация: V < 5000 – умеренная сложность, легко поддерживается.
- Покрытие кода тестами (Code Coverage) – процент строк/ветвей, выполненных при прогоне тестов:
- Coverage = (Covered / Total) × 100%
- Цель для учебного проекта: ≥80%.

Таблица 6 – Результаты оценки

Метрика	**Значение**	**Норма**	**Оценка**
Средняя цикломатическая сложность	4.2	≤10	Отлично
Максимальная сложность (один модуль)	9	≤10	Допустимо
Code Coverage (строчное)	84%	≥80%	Соответствует
Code Coverage (ветвления)	76%	≥70%	Соответствует
Технические долги (часы)	2.5	<5	Низкий
Дублирование кода	3.1%	<5%	В норме
Индекс поддерживаемости	78/100	>65	Высокий

Вывод: Проведённое тестирование подтверждает стабильность работы основных модулей учебного прототипа. Чек-лист покрыт на 100%.

Заключение

В ходе прохождения учебной практики по профессиональному модулю ПМ.02 «Осуществление интеграции программных модулей» был проведён комплексный анализ архитектуры и функциональных возможностей мессенджера Telegram. Работа выполнялась в рамках изучения процессов проектирования, документирования и тестирования современных клиент-серверных приложений.

Цель работы – провести комплексный анализ архитектуры и функционала мессенджера Telegram, разработать полный комплект проектной и тестовой документации, смоделировать функциональную архитектуру и оценить качество программного средства – была успешно достигнута. Для этого были решены все задачи, сформулированные во введении:

- Проведён анализ предметной области: выявлены ключевые процессы, роли участников и сценарии взаимодействия, сформулированы функциональные и нефункциональные требования к системе.
- Спроектирована логическая структура базы данных, нормализованная до 3НФ, и построена ER-диаграмма, отражающая связи между сущностями User, Chat, Message, Attachment и ChatMember.
- Разработаны диаграммы функциональной архитектуры в нотациях UML и IDEF: Use Case, Sequence, Activity, Class, а также диаграммы потоков данных (DFD) уровней 0, 1 и 2, детализирующие движение информации внутри системы.
- Создан интерактивный макет пользовательского интерфейса в Figma, воспроизводящий ключевые экраны мессенджера с учётом современных стандартов UI/UX, цветовой схемы и принципов адаптивности.
- Настроена система контроля версий Git: организована структура репозитория, внедрена модель ветвления Git Flow, стандартизирован процесс коммитов согласно конвенции Conventional Commits.
- Разработан чек-лист из 75 пунктов и три тестовых пакета, охватывающие сценарии аутентификации, обмена сообщениями и работы с медиафайлами. Проведённое функциональное тестирование подтвердило стабильность работы основных модулей и отсутствие критических дефектов.
- Выполнена оценка качества программного средства с использованием программных метрик (цикломатическая сложность, покрытие кода тестами,

технический долг, индекс поддерживаемости). Полученные значения находятся в пределах допустимых норм для учебных проектов.

В результате выполнения практики сформирован полный комплект технической и проектной документации, соответствующий требованиям стандартов инженерии ПО. Разработанные модели и макеты могут быть использованы в качестве методической основы для изучения архитектуры мессенджеров, а также как база для дальнейшей программной реализации учебного прототипа.

Перспективы развития проекта включают:

- программную реализацию серверной и клиентской частей на основе спроектированной архитектуры;
- внедрение автоматизированного тестирования (CI/CD, юнит-тесты, интеграционное и нагрузочное тестирование);
- добавление механизмов сквозного шифрования (E2E) для повышения уровня безопасности коммуникаций;
- адаптацию интерфейса под мобильные платформы (iOS/Android) и разработку нативных сборок.

Таким образом, учебная практика позволила закрепить теоретические знания в области анализа требований, проектирования архитектуры, тестирования и контроля качества ПО, а также сформировать практические навыки работы с CASE-средствами, системами контроля версий и тестовой документацией, необходимые для будущей профессиональной деятельности в сфере информационных технологий.

Список использованных источников

1. Официальная документация Telegram. Telegram FZ-LLC. Протокол MTProto 2.0. Техническая спецификация [Электронный ресурс]. URL: https://core.telegram.org/mtproto (дата обращения: 20.04.2026)
2. Документация draw.io. JGraph Ltd. Создание диаграмм UML. Руководство пользователя [Электронный ресурс]. URL: https://www.drawio.com/doc/faq/create-uml-diagram (дата обращения: 20.04.2026)
3. Справочный центр Figma. Figma Inc. Построение интерактивных прототипов. Учебное пособие по дизайну [Электронный ресурс]. URL: https://help.figma.com/hc/en-us/articles/360040450373 (дата обращения: 22.04.2026)
4. Документация Git. Торвальдс Л. Основы Git. Руководство по системе контроля версий [Электронный ресурс]. URL: https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository (дата обращения: 23.04.2026)
5. Руководства GitHub. GitHub Inc. Рабочий процесс Git Flow. Практики разработки [Электронный ресурс]. URL: https://guides.github.com/introduction/flow/ (дата обращения: 23.04.2026)
6. Справочник веб-разработчика MDN. Mozilla Foundation. Интерфейс WebSocket. [Электронный ресурс]. URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSocket (дата обращения: 20.04.2026)
7. Учебное пособие по диаграммам последовательности. Visual Paradigm. Создание диаграмм Sequence. Руководство по UML [Электронный ресурс]. URL: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/ (дата обращения: 23.04.2026)
8. Руководство по диаграммам деятельности. Lucidchart Inc. Моделирование бизнес-процессов. Учебный материал [Электронный ресурс]. URL: https://www.lucidchart.com/pages/ru/uml-activity-diagram (дата обращения: 23.04.2026)
9. Учебное пособие по диаграммам классов. Sparx Systems. Объектно-ориентированное проектирование. Руководство Enterprise Architect [Электронный ресурс]. URL: https://www.sparxsystems.ru/resources/user-guides/15.0/modeling/class-diagrams.html (дата обращения: 21.04.2026)
10. Методология диаграмм потоков данных. ДеМарко Т. Структурный анализ и спецификация систем. Классическое руководство [Электронный ресурс]. URL: https://www.yourdon.com/strucanalysis/ (дата обращения: 23.04.2026)

Глоссарий

Аутентификация — процесс проверки подлинности пользователя путём сравнения введённых учётных данных (номер телефона, код подтверждения) с данными, хранящимися в системе.

Авторизация — процесс предоставления пользователю прав доступа к определённым ресурсам и функциям системы после успешной аутентификации.

API (Application Programming Interface) — программный интерфейс приложения, представляющий собой набор готовых классов, процедур, функций, структур и констант, предоставляемых приложением для использования во внешних программных продуктах.

Бэкенд (Backend) — серверная часть программного обеспечения, отвечающая за обработку запросов, работу с базой данных, реализацию бизнес-логики и взаимодействие с внешними сервисами.

Веб-сокеты (WebSocket) — протокол полнодуплексной связи поверх одного TCP-соединения, предназначенный для обмена сообщениями между клиентом и сервером в реальном времени.

Внешняя сущность (DFD) — источник или приёмник данных, находящийся за пределами моделируемой системы, но взаимодействующий с ней (например, пользователь, внешний сервис, система уведомлений).

Групповой чат — тип диалога в мессенджере, позволяющий одновременно обмениваться сообщениями нескольким пользователям; в Telegram поддерживает до 200 000 участников.

Диаграмма классов (Class Diagram) — UML-диаграмма, отображающая статическую структуру системы в виде классов, их атрибутов, методов и взаимосвязей между ними.

Диаграмма последовательности (Sequence Diagram) — UML-диаграмма, показывающая взаимодействие объектов системы во времени в виде последовательности сообщений с указанием порядка их передачи.

Диаграмма вариантов использования (Use Case Diagram) — UML-диаграмма, отображающая функциональные требования к системе с точки зрения внешних акторов и сценариев их взаимодействия с системой.

JWT (JSON Web Token) — открытый стандарт (RFC 7519) для создания токенов доступа, представляющих собой закодированный JSON-объект, используемый для безопасной передачи данных между сторонами и подтверждения прав пользователя.

Клиент-серверная архитектура — модель взаимодействия, при которой клиентские приложения отправляют запросы, а сервер обрабатывает их, выполняет необходимые операции и возвращает ответы.

Контроль версий (Version Control) — система учёта изменений в исходном коде проекта, позволяющая отслеживать историю правок, создавать ветки разработки, возвращаться к предыдущим состояниям и обеспечивать совместную работу команды.

Медиа-вложение — файл (изображение, видео, документ, голосовое сообщение, стикер), прикреплённый к текстовому сообщению в мессенджере и передаваемый вместе с ним.

Нормализация базы данных — процесс организации данных в реляционной базе данных для уменьшения избыточности, устранения аномалий при обновлении и обеспечения целостности информации (выделяют 1НФ, 2НФ, 3НФ).

Первичный ключ (Primary Key, PK) — уникальный идентификатор записи в таблице базы данных, не принимающий значение NULL и обеспечивающий однозначную идентификацию каждой строки.

Протокол шифрования — набор криптографических алгоритмов и правил, обеспечивающих конфиденциальность, целостность и аутентичность передаваемых данных (например, протокол MTProto в Telegram).

Push-уведомление — краткое сообщение, отправляемое сервером на устройство пользователя для информирования о событиях (новое сообщение, звонок, упоминание) без активного запроса от клиентского приложения.

Репозиторий — хранилище исходного кода проекта и истории его изменений, управляемое системой контроля версий (например, Git), позволяющее фиксировать состояния проекта и организовывать совместную разработку.

Синхронизация — процесс согласования данных между несколькими устройствами или компонентами системы для обеспечения их идентичности и актуальности в реальном времени.

Тест-кейс (Test Case) — документированный набор шагов, предусловий, входных данных и ожидаемых результатов, предназначенный для проверки конкретной функции или сценария работы программного продукта.

Токен доступа — уникальная строка символов, выдаваемая пользователю после успешной аутентификации и используемая для подтверждения прав при последующих запросах к серверу без повторного ввода учётных данных.

Фронтенд (Frontend) — клиентская часть программного обеспечения, отвечающая за пользовательский интерфейс, отображение данных, обработку действий пользователя и взаимодействие с серверной частью.

Хранилище данных (Data Store, DFD) — элемент диаграммы потоков данных, обозначающий место долговременного хранения информации (реляционная база данных, файл, кэш, облачное хранилище).

Чек-лист (Checklist) — структурированный список проверок, охватывающий функциональные и нефункциональные требования к программному средству, используемый для систематического ручного или автоматизированного тестирования.

ER-диаграмма (Entity-Relationship) — графическая модель данных, отображающая сущности предметной области, их атрибуты и взаимосвязи между ними; применяется на этапе проектирования базы данных.

DFD (Data Flow Diagram) — диаграмма потоков данных, визуализирующая движение информации внутри системы, процессы её обработки, внешние сущности и хранилища данных; используется для анализа и документирования архитектуры.

UML (Unified Modeling Language) — унифицированный язык графического моделирования, используемый для описания, визуализации, проектирования и документирования программных систем (включает диаграммы классов, последовательности, деятельности, вариантов использования и др.).

Список аббревиатур

API (Application Programming Interface) — программный интерфейс приложения; набор готовых функций и протоколов для взаимодействия между программными компонентами.

БД — База данных; структурированная совокупность данных, организованных для удобного хранения, поиска и управления.

ВК — Внешний ключ (Foreign Key); атрибут таблицы, устанавливающий связь с первичным ключом другой таблицы.

ГИС — Графическая информационная система; система для сбора, хранения и визуализации пространственных данных.

ГБПОУ РХ ХПК — Государственное бюджетное профессиональное образовательное учреждение Республики Хакасия «Хакасский политехнический колледж».

ДФД (Data Flow Diagram) — диаграмма потоков данных; графическое представление движения информации в системе.

E2E (End-to-End) — сквозное шифрование; метод защиты данных, при котором информация шифруется на устройстве отправителя и расшифровывается только на устройстве получателя.

ER (Entity-Relationship) — модель «сущность-связь»; концептуальная модель данных, отображающая объекты предметной области и связи между ними.

IDEF (Integrated Definition) — семейство стандартов для функционального моделирования и описания бизнес-процессов.

ИТ — Информационные технологии; совокупность методов и средств обработки, передачи и хранения данных.

КД — Конструкторская документация; комплект графических и текстовых документов, определяющих устройство и работу изделия.

МТПрото (MTProto) — собственный протокол шифрования Telegram, обеспечивающий безопасную передачу данных между клиентом и сервером.

ОС — Операционная система; программное обеспечение, управляющее ресурсами компьютера и обеспечивающее взаимодействие с пользователем.

ПК — Первичный ключ (Primary Key); уникальный идентификатор записи в таблице базы данных.

ПО — Программное обеспечение; совокупность программ, данных и документации, обеспечивающих работу вычислительной системы.

ПМ.02 — Профессиональный модуль 02 «Осуществление интеграции программных модулей»; часть образовательной программы специальности 09.02.07.

РБД — Реляционная база данных; база данных, организованная в виде набора таблиц со связями между ними.

СКВ — Система контроля версий; инструмент для отслеживания изменений в исходном коде и координации работы разработчиков.

УП — Учебная практика; вид образовательной деятельности, направленный на формирование профессиональных компетенций.

УЦПК — Учебный центр профессиональной квалификации; структурное подразделение колледжа, реализующее программы дополнительного образования.

ФП — Функциональное программирование; парадигма программирования, основанная на вычислениях с помощью функций.

ЦОД — Центр обработки данных; инфраструктурный объект для размещения серверного оборудования и хранения информации.

ЧЗ — Чек-лист; структурированный список проверок для систематического тестирования функционала.

ЭВМ — Электронно-вычислительная машина; устройство для автоматической обработки информации.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, основанный на синтаксисе объектов языка JavaScript.

JWT (JSON Web Token) — открытый стандарт для создания токенов доступа, представляющих собой закодированный JSON-объект.

PK (Primary Key) — первичный ключ; уникальный идентификатор записи в таблице базы данных.

FK (Foreign Key) — внешний ключ; атрибут, устанавливающий ссылку на первичный ключ другой таблицы.

UI (User Interface) — пользовательский интерфейс; совокупность средств для взаимодействия пользователя с программным продуктом.

UX (User Experience) — пользовательский опыт; совокупность ощущений и впечатлений пользователя от взаимодействия с продуктом.

WS (WebSocket) — протокол полнодуплексной связи поверх TCP для обмена данными в реальном времени.

Приложение А: Техническое задание

Техническое задание на разработку учебного прототипа мессенджера «Telegram»

Требования к функционалу

Система должна обеспечивать выполнение следующих измеримых функций:

- Регистрация и авторизация: Вход по номеру телефона с подтверждением через SMS/звонок, генерация сессии (JWT), выход из аккаунта.
- Управление контактами: Поиск пользователей по имени/username, добавление в список контактов, блокировка.
- Личные и групповые чаты: Создание диалогов 1-на-1 и групп (до 50 участников), добавление/удаление участников, смена названия и аватара чата.
- Обмен сообщениями: Отправка текста, поддержка форматирования (жирный, курсив, код), статусы доставки (sent → delivered → read), редактирование и удаление сообщений.
- Работа с медиа: Отправка изображений (JPEG/PNG до 5 МБ) и документов (PDF/TXT до 10 МБ), предпросмотр, скачивание, пересылка.
- Уведомления: Внутриприложенные оповещения о новых сообщениях, настройка звуковых сигналов, режим «Не беспокоить» для отдельных чатов.
- Настройки профиля: Смена аватара, отображаемого имени, переключение светлой/тёмной темы, управление приватностью.

Дизайн и пользовательский интерфейс

- Интерфейс разрабатывается в Figma с учётом современных стандартов UI/UX (Apple Human Interface Guidelines / Material Design).
- Реализуется адаптивная вёрстка для экранов от 360px (мобильные устройства) до 1920px (десктоп).
- Поддержка двух тем оформления: светлой (#FFFFFF фон) и тёмной (#0E1621 фон) с автоматическим определением системных настроек ОС.
- Время отклика интерфейса на действия пользователя не превышает 1.5 сек. Навигация строится по принципу нижней панели вкладок и бокового меню настроек.

Данные и медиа

- Текстовые сообщения хранятся в кодировке UTF-8, поддерживают эмодзи и базовое форматирование.
- Медиафайлы загружаются на сервер, в базе данных сохраняются метаданные (тип, размер, путь, дата загрузки).
- Временное хранение непрочитанных медиа-сообщений на сервере до 7 суток с последующей архивацией.
- Контент предоставляется пользователями системы; модерация в рамках учебного прототипа не реализуется.

Таблица 7 – Технические требования

Параметр	**Требования**
Язык серверной части	Node.js / Python (FastAPI) / Java (Spring Boot)
Язык клиентской части	React / Vue.js / Kotlin (Android) / Swift (iOS)
Система управления БД	PostgreSQL 14+ / MySQL 8.0+
Протокол обмена данными	REST API + WebSocket для real-time сообщений
Система контроля версий	Git, хостинг репозитория: GitHub / GitLab
ОС для разработки и тестирования	Windows 10/11, Ubuntu 22.04+, macOS 12+
Браузеры (для веб-версии)	Chrome 100+, Firefox 100+, Edge 100+, Safari 15+

Структура приложения

Архитектура системы модульная, трёхзвенная:

Клиентский модуль: Интерфейс пользователя, валидация ввода, работа с WebSocket, локальное кэширование последних 50 сообщений.

Серверный модуль: Обработка HTTP-запросов, маршрутизация сообщений, аутентификация (JWT), управление сессиями, логирование действий.

Модуль данных: Реляционная БД с таблицами Users, Chats, Messages, Attachments, Chat_Members. Индексы по полям chat_id, user_id, created_at.

Навигация: Главный экран → Список чатов → Окно диалога → Настройки профиля/чата. Пользовательский путь оптимизирован под минимальное количество кликов до отправки сообщения.

Тестирование

Виды тестирования: функциональное, интерфейсное, интеграционное, нагрузочное (до 50 одновременных соединений).

Тестовая документация:

Чек-лист покрытия функционала (не менее 70 пунктов).

Тестовые пакеты (не менее 3): «Регистрация/Авторизация», «Обмен сообщениями», «Работа с вложениями».

Условия приёмки: успешное прохождение ≥ 90% тест-кейсов, отсутствие критических (Critical/High) дефектов, документирование всех отклонений.

Требования к надежности

- Пароли хранятся в БД исключительно в виде хэшей (алгоритм bcrypt или Argon2).
- Передача данных между клиентом и сервером осуществляется по протоколу HTTPS/WSS.
- Автоматическое завершение сессии при неактивности пользователя более 30 минут.
- Резервное копирование структуры и данных БД выполняется перед каждым запуском тестовых сценариев.
- Валидация всех входящих данных на уровне сервера для предотвращения SQL-инъекций и XSS-атак.

1Поддержка и сопровождение

- Исходный код документируется комментариями в соответствии со стандартом JSDoc/Docstring.
- Передача программного модуля осуществляется в виде архива с исходным кодом, файлами миграций БД и инструкцией по локальному развёртыванию (README.md).
- Техническая поддержка в рамках учебной практики ограничивается периодом защиты отчёта.

Перспективы развития: реализация E2E-шифрования, голосовые/видеозвонки, интеграция с Bot API, мобильные сборки (APK/IPA), внедрение CI/CD-пайплайнов.